

ELCO-Dealer
Guía software
Electrónica de Consumo

JAVIER CARRETERO,
CARLOS GONZÁLEZ,
ITZIAR OTAÑO,
ÁLVARO POVEDA,
CRISTINA RODRÍGUEZ,
JUAN SALGADO

Universidad Politécnica de Madrid
28 de abril de 2026



Índice

1. Introducción	2
2. Objetivo y alcance	2
3. Arquitectura del software embebido	2
3.1. Criterios de diseño	2
3.2. Estructura del repositorio	3
3.3. Mapa de pines y elementos controlados	3
3.4. Máquina de estados finitos	4
3.5. Gestión de eventos de usuario	5
3.6. Control de sensores	5
3.7. Programación de los motores	6
3.8. Timeouts y recuperación de errores	6
3.9. Modo de diagnóstico	7
3.10. Compilación y carga	7
3.11. Parámetros de calibración	8
3.12. Flujo de uso desde software	8
4. Página web	9
4.1. Arquitectura y Contenidos de la Plataforma	9
4.2. Estrategia de Captación: Wishlist y Reserva	10
4.3. Integración con la Filosofía Open Source	11
5. Conclusión	12



1. Introducción

La presente guía documenta el software desarrollado para el prototipo educativo **ELCO-Dealer**, realizado en el marco de la asignatura **Electrónica de Consumo** del Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación de la Universidad Politécnica de Madrid.

El objetivo del documento es explicar de forma ordenada la arquitectura del código, la lógica de control implementada en Arduino y la relación entre el software embebido, los sensores, los motores y la interfaz de usuario. Además, se incluye una descripción de la plataforma web asociada al proyecto, concebida como escaparate técnico y comercial del sistema.

Esta guía complementa al manual de usuario y montaje: mientras aquel se centra en el ensamblaje y operación física del prototipo, este documento describe cómo está organizado el programa, qué responsabilidades tiene cada módulo y qué parámetros deben revisarse durante la calibración.

2. Objetivo y alcance

La guía está orientada a facilitar el mantenimiento y la ampliación del software del **ELCO-Dealer**. No pretende sustituir al código fuente, sino proporcionar una lectura técnica del mismo: qué hace cada módulo, cómo se comunican entre sí y qué decisiones de diseño permiten que el prototipo funcione de forma determinista.

El software se ha desarrollado en **C++ para Arduino** y se ejecuta sobre una **Arduino Mega 2560 R3**. La implementación se basa en una separación estricta entre lógica de estados, acceso a hardware, interfaz de usuario y depuración. Esta decisión reduce el acoplamiento entre partes del sistema y permite probar cada subsistema de forma independiente antes de operar la máquina completa.

3. Arquitectura del software embebido

3.1. Criterios de diseño

El programa se ha diseñado siguiendo una arquitectura modular, de forma que cada archivo tenga una responsabilidad concreta y no se mezclen decisiones de alto nivel con detalles eléctricos. Esta separación resulta especialmente importante en un prototipo electromecánico, ya que permite modificar un pin, ajustar un tiempo de rebote o cambiar un umbral de la botonera sin alterar la lógica general de reparto.

Los criterios principales han sido los siguientes:

- **Determinismo:** el sistema debe responder siempre de la misma forma ante la



misma secuencia de eventos.

- **No bloqueo:** el código evita esperas con `delay()` y trabaja mediante actualizaciones periódicas basadas en `millis()`.
- **Separación por capas:** la FSM decide qué debe ocurrir, pero no manipula pines directamente.
- **Seguridad de operación:** los movimientos de giro y empuje incorporan *timeouts* para evitar que un motor quede funcionando indefinidamente ante un fallo de sensor.
- **Calibración centralizada:** pines, umbrales, tiempos y velocidades PWM se agrupan en `src/config.h`.

3.2. Estructura del repositorio

El punto de entrada del programa es mínimo. El archivo `card_dealer.ino` únicamente inicializa la máquina de estados en `setup()` y la actualiza de forma continua en `loop()`. El comportamiento real se delega en los módulos de `src/`.

Archivo	Responsabilidad
<code>card_dealer.ino</code>	Entrada principal de Arduino. Llama a <code>fsm_init()</code> y <code>fsm_update()</code> .
<code>src/config.h</code>	Define pines, límites de jugadores, tiempos de seguridad, PWM, polaridad de sensores y umbrales de la botonera.
<code>src/config_checks.h</code>	Verifica en compilación que no existan duplicados de pines ni incoherencias básicas de configuración.
<code>src/fsm.*</code>	Implementa la lógica del sistema mediante una máquina de estados finitos. Decide cuándo girar, empujar cartas, volver a HOME o entrar en error.
<code>src/hardware.*</code>	Centraliza el acceso a motores DC, drivers L298N, sensor Hall HOME y sensores digitales de carta.
<code>src/ui.*</code>	Gestiona la pantalla LCD 16x2 y la botonera analógica conectada a A0. Traduce pulsaciones físicas en eventos lógicos.
<code>src/diagnostics.*</code>	Permite probar sensores y motores de forma independiente durante el montaje y calibración.
<code>src/debug.*</code>	Encapsula los mensajes por <code>Serial</code> mediante macros de depuración.

Cuadro 1: Organización del software embebido del ELCO-Dealer

3.3. Mapa de pines y elementos controlados

El mapa de pines se encuentra en `Config` y actúa como única fuente de verdad. Esta decisión evita que aparezcan números de pin dispersos por el código y facilita la revisión del cableado.



Elemento	Pines configurados
LCD 16x2	D4–D7 en 4, 5, 6 y 7; RS en 8; EN en 9; retroiluminación en 10.
Botonera LCD Keypad	Entrada analógica A0, interpretada mediante umbrales de tensión.
L298N 1	PWM en 11 y 12; dirección en 22, 23, 24 y 25.
L298N 2	PWM en 2 y 3; dirección en 26, 27, 28 y 29.
Sensor de carta activo	CARD_SENSOR_1 en el pin 30.
Sensor HOME	Sensor Hall en el pin 33.

Cuadro 2: Resumen del mapa de pines definido en `src/config.h`

La versión actual del software documenta un sensor de carta activo en `CARD_SENSOR_1`. El diseño deja preparada la ampliación a más sensores, pero cualquier activación adicional debe hacerse modificando `CARD_SENSOR_COUNT`, el vector de pines y las comprobaciones de compilación correspondientes.

3.4. Máquina de estados finitos

La FSM es el núcleo del sistema. Su función es coordinar la secuencia de operación del prototipo sin acceder directamente a pines ni duplicar lógica de hardware. Cada transición se produce por un evento de usuario, por la detección de un flanco de sensor o por un *timeout* de seguridad.

Estado	Función dentro del sistema
OFF	Estado seguro inicial. Todos los motores están detenidos y la pantalla indica que se mantenga pulsado SELECT.
INIT	Inicialización lógica tras encendido. Reinicia contadores y prepara el menú de configuración.
SET_PLAYERS	Selección del número de jugadores, con límite máximo de 6.
SET_CARDS	Selección del número de cartas por jugador, con límite definido por CMAX.
AUTO_INIT	Preparación del reparto automático. Reinicia el progreso de la ronda.
ROTATE_AUTO	Gira la plataforma hasta el slot objetivo del jugador actual.
PUSH_AUTO	Acciona el motor de reparto hasta detectar una carta mediante el sensor activo.
RETURN_HOME_AUTO	Devuelve el sistema a la posición HOME tras completar la ronda automática.
IDLE_HOME	Estado de reposo operativo. Permite lanzar otra ronda o repartir una carta manual.
ROTATE_ONE	Gira hacia el siguiente jugador en modo manual.
PUSH_ONE	Reparte una única carta y espera su detección.
RETURN_HOME_MANUAL	Vuelve a HOME tras el reparto manual.
ERROR	Estado seguro ante fallo de giro, carta o retorno a HOME. Los motores se detienen.

Cuadro 3: Estados principales implementados en `src/fsm.cpp`



La distribución de jugadores se calcula sobre 6 posiciones lógicas, correspondientes a los imanes o slots de referencia. Para cada jugador, la función de cálculo asigna un slot objetivo proporcional al número total de jugadores configurado:

$$\text{slot objetivo} = \left\lfloor \frac{\text{índice de jugador} \cdot 6}{\text{número de jugadores}} \right\rfloor$$

Con esta estrategia se mantiene una separación uniforme entre jugadores y se evita invertir el sentido de giro. La plataforma gira siempre en el mismo sentido, y el primer imán detectado durante un retorno a HOME redefine el origen lógico como slot 0.

3.5. Gestión de eventos de usuario

La interfaz física se basa en la botonera de una LCD Keypad Shield cableada manualmente. El módulo `ui.cpp` lee el valor analógico de A0 y lo transforma en botones físicos. En la versión actual se utilizan dos acciones lógicas:

- **PWR:** asociado al botón SELECT. Una pulsación corta confirma o avanza; una pulsación larga enciende o apaga lógicamente el sistema.
- **ADD:** asociado al botón RIGHT. Incrementa jugadores, cartas o lanza una carta manual según el estado.

La detección de pulsaciones utiliza un tiempo de antirrebote de `UI_DEBOUNCE_MS` y diferencia pulsación corta y larga mediante `UI_LONG_PRESS_MS`. Internamente, los eventos se priorizan para que `PWR_LONG` tenga precedencia sobre eventos de menor importancia.

3.6. Control de sensores

Los sensores digitales se gestionan exclusivamente desde `hardware.cpp`. Cada sensor mantiene su propio estado estable, último valor leído, instante del último cambio, instante del último evento y banderas de flanco ascendente o descendente.

El sensor Hall HOME utiliza:

- lectura digital con nivel activo configurable en `HOME_SENSOR_ACTIVE_LEVEL`;
- antirrebote mediante `HALL_DEBOUNCE_MS`;
- tiempo muerto mediante `HALL_DEAD_TIME_MS`;
- rearme lógico en la FSM mediante `HALL_REARM_MS`.

El sensor de carta sigue la misma filosofía, pero con tiempos específicos definidos como `CARD_SENSOR_DEBOUNCE_MS`, `CARD_SENSOR_DEAD_TIME_MS` y `CARD_SENSOR_REARM_MS`. De esta forma se evita contar varias veces una misma carta cuando el sensor permanece activo durante el paso físico del naípe.



3.7. Programación de los motores

Todos los motores DC se controlan desde la capa Hardware. La FSM no escribe pines directamente: solicita acciones lógicas como `motor_forward()`, `motor_stop()` o `motors_stop_all()`.

Cada canal de motor se representa mediante:

- un pin PWM conectado al enable del L298N;
- dos pines de dirección;
- una velocidad PWM actual;
- una dirección lógica.

El motor de rotación se controla como `MotorId::ROTATION` y los motores de reparto como `DEAL_1`, `DEAL_2` y `DEAL_3`. En la lógica actual de reparto se utiliza `DEAL_1` como expulsor principal, manteniendo los demás canales disponibles para pruebas o futuras ampliaciones mecánicas.

La velocidad inicial de giro se define en `ROTATION_PWM` y la velocidad de reparto en `DEAL_PWM`. Estos valores deben calibrarse sobre el prototipo real, ya que dependen de la fricción de los rodillos, el peso de la baraja, la alimentación de los motores y la alineación del mecanismo.

3.8. Timeouts y recuperación de errores

Para que el sistema sea tolerante a fallos físicos, las acciones con movimiento tienen un tiempo máximo de ejecución:

Parámetro	Uso
<code>ROTATE_TIMEOUT_MS</code>	Tiempo máximo permitido para alcanzar un slot durante el giro.
<code>PUSH_TIMEOUT_MS</code>	Tiempo máximo para detectar una carta tras activar el motor de reparto.
<code>RETURN_HOME_TIMEOUT_MS</code>	Tiempo máximo para volver a detectar HOME durante el retorno.

Cuadro 4: Timeouts principales de seguridad

Si se supera alguno de estos límites, la FSM entra en `ERROR`, detiene todos los motores y muestra el tipo de fallo en la pantalla. Desde este estado se puede iniciar una recuperación mediante una pulsación corta de `PWR`, que ordena un retorno a `HOME`.



3.9. Modo de diagnóstico

El modo de diagnóstico permite validar el prototipo antes de ejecutar la FSM completa. Se activa en compilación mediante la macro:

```
CARD_DEALER_DIAGNOSTIC_MODE=true
```

En este modo, el usuario navega por páginas de prueba usando la pantalla LCD:

- **PWR corto:** avanzar a la siguiente página de diagnóstico.
- **ADD corto:** activar o detener el motor mostrado en pantalla.
- **PWR largo:** parada segura y vuelta a la página HOME.

El recorrido recomendado es comprobar primero el sensor HOME, después el sensor de carta activo y finalmente cada motor. No debe pasarse a modo normal hasta que los flancos de sensor sean repetibles y ningún motor quede activado al cambiar de pantalla.

3.10. Compilación y carga

El proyecto está preparado para compilarse con Arduino CLI usando la placa **Arduino Mega**. En modo normal:

```
arduino-cli compile --fqbn arduino:avr:mega .
```

Para compilar el modo de diagnóstico sin modificar el archivo de configuración:

```
arduino-cli compile --fqbn arduino:avr:mega '  
  --build-property compiler.cpp.extra_flags=-DCARD_DEALER_DIAGNOSTIC_MODE=true .
```

La carga en placa debe realizarse indicando el puerto serie real detectado en el equipo:

```
arduino-cli upload -p COMx --fqbn arduino:avr:mega .
```

Antes de cargar una versión de funcionamiento normal es recomendable revisar que `CARD_DEALER_DIAGNOSTIC_MODE` esté desactivado o que la compilación no esté sobrescribiendo dicha macro.



3.11. Parámetros de calibración

La calibración final debe hacerse con el prototipo físico montado. Los parámetros más sensibles se encuentran en `src/config.h`:

Grupo	Parámetros a revisar
Botonera	Umbrales KEYPAD_RIGHT_MAX, KEYPAD_UP_MAX, KEYPAD_DOWN_MAX, KEYPAD_LEFT_MAX y KEYPAD_SELECT_MAX.
Sensores	Polaridad CARD_SENSOR_ACTIVE_LEVEL y HOME_SENSOR_ACTIVE_LEVEL.
Filtrado	Tiempos de antirrebote, tiempo muerto y rearme para Hall y sensor de carta.
Motores	ROTATION_PWM y DEAL_PWM.
Seguridad	ROTATE_TIMEOUT_MS, PUSH_TIMEOUT_MS y RETURN_HOME_TIMEOUT_MS.

Cuadro 5: Parámetros software que requieren calibración física

Un ajuste correcto debe conseguir que el sensor de carta genere un único evento por carta, que HOME se detecte una sola vez por imán y que los motores tengan margen suficiente para mover la mecánica sin golpes bruscos.

3.12. Flujo de uso desde software

El flujo normal del programa se puede resumir del siguiente modo:

1. El sistema arranca en OFF con todos los motores detenidos.
2. Una pulsación larga de SELECT pasa a INIT.
3. En SET_PLAYERS, RIGHT incrementa el número de jugadores y SELECT confirma.
4. En SET_CARDS, RIGHT incrementa las cartas por jugador y SELECT inicia el reparto automático.
5. En reparto automático, la FSM alterna entre giro y empuje hasta completar todas las cartas.
6. Al terminar, el sistema vuelve a HOME y queda en IDLE_HOME.
7. Desde IDLE_HOME, SELECT repite un reparto automático y RIGHT reparte una carta individual.

Este flujo coincide con la experiencia prevista para el usuario final, pero mantiene internamente los controles de seguridad y recuperación necesarios para un prototipo físico.



4. Página web

Como pilar fundamental de la estrategia de difusión y comercialización del proyecto, se ha diseñado una plataforma web propia accesible en la dirección: <https://fantastic-douhua-aab9a0.netlify.app/>. Siguiendo la filosofía de la asignatura de actuar como una “pequeña empresa de ingeniería”, la web no solo sirve como escaparate comercial, sino también como un centro interactivo para la captación de inversores y la distribución centralizada de la documentación técnica.

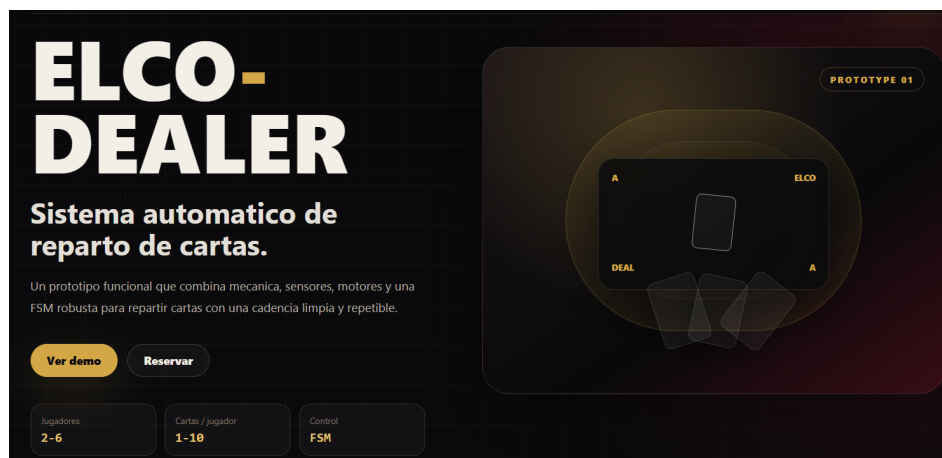


Figura 1: Interfaz principal de la web ELCO-Dealer con acceso a la Demo y reserva del prototipo.

4.1. Arquitectura y Contenidos de la Plataforma

La interfaz ha sido diseñada para segmentar la información en bloques estratégicos, facilitando la comprensión del producto tanto a nivel funcional como técnico:

- **Highlights y Demo:** Esta sección presenta la propuesta de valor del **ELCO-Dealer**. Incluye un video demostrativo alojado en YouTube que documenta el funcionamiento real del prototipo, su capacidad de barajado automático y el modo de uso para el cliente final (ver Figura 1).
- **Simulador de Reparto:** Herramienta interactiva (Figura 2) que permite al usuario visualizar de forma lógica el proceso de distribución de naipes. Este simulador emula el comportamiento del software embebido en la Arduino Mega 2560, manteniendo los límites reales del prototipo (máximo 6 jugadores).
- **Engineering (Arquitectura de Software):** Se detalla el diseño orientado a la robustez (Figura 3), destacando el uso de una **Máquina de Estados Finitos (FSM)** para gestionar la lógica de control. La web explica cómo el sistema evita bloqueos mediante la implementación de prioridades de eventos, *timeouts* y transiciones claras sin acceso directo a pines para asegurar la portabilidad del código.



- **Hardware y Sensórica:** Bloque técnico que describe la integración de los componentes clave detallados en el listado de materiales (BOM). Se resalta el uso de los drivers **L298N** para el control de potencia [1], el sensor de efecto Hall **A3144** para la autocalibración de 0 grados y los sensores infrarrojos **TCRT5000** para la detección de cartas por flancos.
- **User Interface (UI):** Detalla el subsistema de configuración. El usuario puede definir interactivamente el número de jugadores y el modo de reparto a través de la pantalla y botonera integradas.

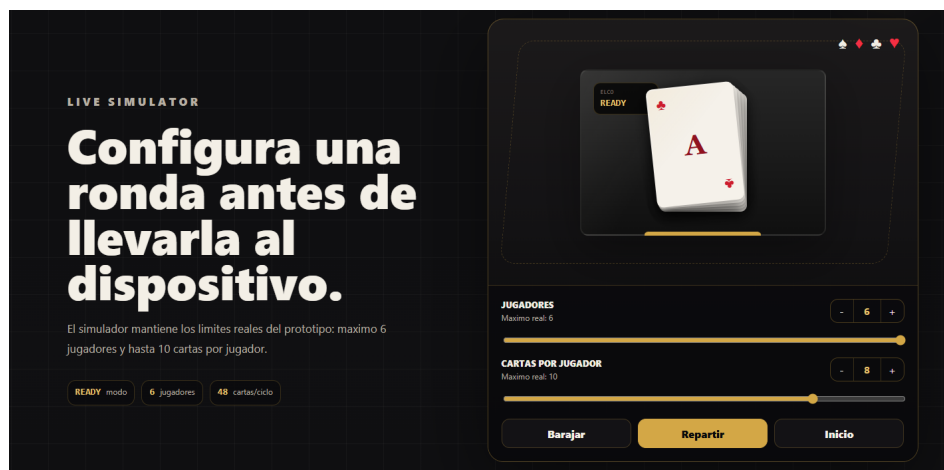


Figura 2: Detalle del simulador lógico de barajado y reparto integrado en la plataforma.



Figura 3: Detalle de la FSM, el hardware y la UI del prototipo.

4.2. Estrategia de Captación: Wishlist y Reserva

Siguiendo un enfoque honesto de lanzamiento de producto bajo el contexto de un proyecto de ingeniería universitaria, la web incluye una sección de reserva (Figura 4). Esta funcionalidad permite registrar el interés de posibles clientes y construir un *storytelling* creíble alrededor del prototipo, alineándose con la viabilidad económica planteada en la memoria del proyecto.



The screenshot displays a dark-themed web interface for 'ELCO-DEALER'. On the left, a 'RESERVE' section features the headline 'A polished concept, ready to be demonstrated.' and a paragraph explaining that the product is a prototype. Below this are two buttons: 'Join the Waitlist' and 'Watch Demo'. On the right, a 'Concept reservation' form is shown. It includes a title 'ELCO-DEALER Prototype Access', a list of features (University engineering prototype, Functional motion and sensing system, Designed for future public demo assets), a disclaimer about public release, and input fields for 'NOMBRE' (Tu nombre) and 'EMAIL' (tu@email.com). There is also a dropdown for 'INTERES' (Comprar una unidad) and a final 'Unirme a wishlist' button.

Figura 4: Formulario de reserva y wishlist integrado en la plataforma web.

4.3. Integración con la Filosofía Open Source

En cumplimiento con los requisitos de documentación pública de la asignatura, el pie de página de todas las secciones incluye enlaces directos a los resultados publicados en redes de distribución técnica:

- **GitHub:** Repositorio con el código fuente completo y pruebas de módulos [2].
- **Instructables:** Guía paso a paso para la replicación del proyecto por parte de la comunidad [3].



5. Conclusión

El software del **ELCO-Dealer** se ha planteado como una base robusta para un prototipo físico sometido a tolerancias mecánicas, posibles rebotes de sensores y pequeñas variaciones en el movimiento de cartas. La combinación de una máquina de estados finitos, una capa de hardware centralizada, eventos de usuario priorizados y un modo de diagnóstico permite que el sistema sea comprensible, verificable y ampliable.

Como líneas de mejora software se propone ampliar la capa de sensórica a varios puntos de detección de carta, registrar métricas de fallo durante el uso real, añadir perfiles de calibración por tipo de baraja y conectar la electrónica con la plataforma web o una aplicación móvil en futuras versiones del producto.



Referencias

1. STMicroelectronics, *L298N Dual Full-Bridge Driver*, 2019, datasheet oficial. [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>
2. Equipo ELCO-Dealer, "Repositorio github del proyecto elco-dealer," 2026, repositorio público del código fuente.
3. —, "Guía instructables del proyecto elco-dealer," 2026, guía pública de montaje y replicación.
4. Arduino, *Arduino Mega 2560 Rev3*, 2024, datasheet oficial. [Online]. Available: <https://docs.arduino.cc/hardware/mega-2560/>
5. STMicroelectronics, *L298N Dual Full-Bridge Driver*, 2019, datasheet oficial. [Online]. Available: <https://www.st.com/resource/en/datasheet/l298.pdf>
6. Adafruit Industries, *DC Gearbox Motor - "TT Motor 200RPM - 3 to 6VDC*, 2018. [Online]. Available: <https://www.adafruit.com/product/3777>
7. Adafruit, *Slip Ring 22mm Diameter, 6 Wires, 2A (SRC022A-6)*, 2020. [Online]. Available: <https://www.adafruit.com/product/736>
8. Vishay Semiconductors, *TCRT5000 Reflective Optical Sensor*, 2009. [Online]. Available: <https://www.vishay.com/docs/83760/tcrt5000.pdf>
9. Allegro MicroSystems, *A3144 Sensitive Hall-Effect Switch*, 2005. [Online]. Available: <https://www.alldatasheet.com/datasheet-pdf/pdf/55092/ALLEGRO/A3144.html>
10. DFRobot, *DFRobot LCD Keypad Shield (DFR0009)*, 2011. [Online]. Available: https://wiki.dfrobot.com/LCD_KeyPad_Shield_For_Arduino_SKU_DFR0009
11. "Automatic card shuffler machine," YouTube, 2026, video consultado el 4 de marzo de 2026.
12. "Card dealer machine prototype," YouTube, 2026, video consultado el 4 de marzo de 2026.
13. "Automatic card dispenser system," YouTube, 2026, video consultado el 4 de marzo de 2026.
14. "Easy to make - motorized lazy susan," <https://www.youtube.com/watch?v=kJvAgkTQ9qY>, 2020, accedido el 15 de abril de 2026.
15. Arduino, *Arduino Mega 2560 Rev3*, 2024, datasheet oficial. [Online]. Available: <https://docs.arduino.cc/hardware/mega-2560/>